Security Audit Report

BeeZee (BZE)

Phase 1

v1.0

March 18, 2026

Table of Contents

License

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by BeeZee to perform a security audit of BeeZee (BZE).

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/bze-alphateam/bze
Commit	`d6c1b6358cb2b20759ff14dba3b359fa028affeb`
Scope	The scope is restricted to the following folders: • `app` (excluding `app/upgrades`) • `x/rewards` • `x/tradebin` • `x/txfeecollector`

Fixes verified at commit	`cc77b51fcbb7d0de61fa05a8622165aa95b729ee` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

BeeZee is a Cosmos SDK based blockchain that integrates on-chain trading, incentive distribution, and fee management at the protocol level.

The tradebin module implements the trading subsystem: it maintains markets and liquidity pools, computes order requirements, executes swaps using a constant-product AMM, updates pool reserves, and splits swap fees between liquidity providers, a fee-collector account, and a burn module.

The rewards module manages both trading-reward campaigns and staking-reward programs by storing program parameters, tracking participation and leaderboards, computing payouts per epoch, updating reward state, and burning unclaimed or expired reward balances.

The txfeecollector module aggregates fees and other balances held by modules, determines which non-native assets can be swapped into the native denomination using available liquidity from the trading module, performs those swaps when possible, and then routes the resulting native tokens to the global fee collector, the community pool, or the burner module according to predefined rules.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	The cosmos appchain includes multiple modules with hooks and ABCI methods.
Code readability and clarity	Medium-High	-
Level of documentation	Medium-High	-
Test coverage	Low	Test coverage cannot be computed due to errors during test execution.

Summary of Findings

No	Description	Severity	Status
1	Unbounded denom accumulation can impact ABCI functionality	Critical	Resolved
2	Unbounded pending unlock iteration in epoch hook allows consensus halting Denial of Service attack	Critical	Resolved
3	Duplicate cancel orders allow order book Denial of Service attacks via processing batch exhaustion	Major	Resolved
4	Lack of deduplication in `FillOrders` enables order book queue flooding	Major	Resolved
5	Crossed book state allows griefing by blocking new sell orders	Major	Resolved
6	Unchecked staking reward duration addition can result in stranded funds	Major	Resolved
7	Unbounded reward hooks in epoch `BeginBlocker` enable Denial of Service	Major	Resolved
8	Trading rewards only validate order book markets, excluding AMM pools	Major	Resolved
9	Rounding-induced lockup risk in claimPending enabling temporary denial of exit	Major	Resolved
10	Fee denomination context not set during `ReCheckTx` causing mempool evictions for non-native fees	Major	Resolved
11	Validation allows the creation of a liquidity pool with one empty asset	Minor	Resolved
12	Takers of queued opposite-side liquidity can pay the maker fee rate	Minor	Acknowledged
13	Inconsistent zero-reward handling enables no-op claim-event spam	Minor	Resolved
14	Delimiter-based key encoding allows cross-market prefix pollution	Minor	Resolved
15	Trading reward expiration uses a hardcoded period instead of the user-defined duration field	Minor	Resolved

16	Multiple pending trading reward activations for the same market lead to orphaned rewards	Minor	Resolved
17	Missing duration cap during staking reward updates allows bypass of the maximum duration constraint	Minor	Resolved
18	Break statement in order filling skips potentially fillable orders at the same price level	Minor	Resolved
19	Pending trading reward removal commits partial state when burn operation fails	Minor	Resolved
20	`JoinStaking` allows joining expired or finished rewards	Minor	Acknowledged
21	Global queue counter never resets while failed messages persist	Minor	Acknowledged
22	Genesis validation only checks params, allowing malformed state import	Minor	Acknowledged
23	Time-of-check-time-of-use gap in deep liquidity validation allows overpayment	Minor	Resolved
24	Function returns partial swap results on error without context flush	Minor	Resolved
25	Trading reward distribution failures are silently ignored	Minor	Resolved
26	Burning IBC voucher tokens can lead to accounting discrepancies	Minor	Resolved
27	Weak error handling in `CalculateMinAmount` may bypass minimum order protections	Minor	Resolved
28	Deprecated `x/crisis` module contains unpatched vulnerabilities with no upstream fix	Informational	Resolved
29	Incorrect amount tracked in liquidity addition event emits misleading data	Informational	Resolved
30	Pervasive use of `string` types for numeric and coin parameters bypasses Go type safety across all modules	Informational	Acknowledged
31	Duplicate validation in multi-swap	Informational	Resolved
32	Updating `ValidatorMinGasFee` to a non-native denomination breaks the fee validation logic	Informational	Resolved
33	Misleading variable naming in `MultiSwap` swap	Informational	Resolved

	loop		
34	Unused `ErrSample` error variable	Informational	Resolved
35	Silent error handling in `transformPrice` enables state corruption via malformed genesis import	Informational	Resolved
36	Misleading error messages	Informational	Resolved
37	Format string missing denom parameter in error message	Informational	Resolved
38	Magic string used for context key instead of a defined constant	Informational	Resolved
39	Missing defensive validation for negative `amountTraded` in hook function	Informational	Resolved
40	Missing validation for negative reserves could allow state corruption	Informational	Resolved
41	Misleading comment may cause confusion	Informational	Resolved
42	Variable assignment is immediately overwritten	Informational	Resolved
43	Invalid creator address errors are mislabeled as unauthorized	Informational	Resolved
44	Pending unlock accumulation logic runs unnecessarily for zero-lock rewards	Informational	Resolved
45	Invalid amount comparison silently uses undefined behavior	Informational	Resolved
46	Multi-swap route length error message incorrectly implies that zero routes are valid	Informational	Resolved
47	Missing validation for distribution amount	Informational	Resolved
48	Staking reward duration parsing is inconsistent between validation and execution	Informational	Resolved
49	Missing `nil` check on optional `TradingKeeper` dependency leading to potential runtime panic	Informational	Resolved

Detailed Findings

1. Unbounded denom accumulation can impact ABCI functionality

Severity: Critical

In the `convertFeesAndSend` function in `x/txfeecollector/keeper/service_fees_handler.go:45`, when the `sendSkipped` parameter is `false`, denoms that cannot be swapped and converted to the native denom can potentially accumulate and never be removed from the module account.

Specifically, the `txfeecollector` module defines `sendSkipped` as `false`. It is important to note that the `txfeecollector` module is not in the `blockAccAddrs` in the application configuration, so it can receive funds.

Each time this function calls `convertFees`, it retrieves all balances of the module account via `GetAllBalances` and iterates over every coin, performing multiple store reads per denom (`IsNativeDenom`, `CanSwapForNativeDenom`, `HasDeepLiquidityWithNativeDenom`). When a coin has no liquidity pool with the native denom, it is categorized as `nonSwapable` and returned in the skipped result. However, because `ConvertCollectedFeesToNativeDenom` passes `sendSkipped=false`, these coins are silently discarded from the return value and remain in the module account.

Since the `txfeecollector` module account is not included in `blockAccAddrs` (defined in `app/app_config.go:209-218`), any user can send arbitrary denoms to the module account via `MsgSend`.

An attacker could exploit this by sending small amounts of hundreds of denoms to the `txfeecollector` module account. These denoms would permanently reside in the account and be iterated over in every `EndBlock`, with each denom requiring multiple KV store lookups to determine it is not swappable. As the number of denoms grows, the computational cost of `EndBlock` increases linearly, potentially degrading block times and overall chain performance.

The `BurnerFeeCollector` and `FeeCollector` module accounts are partially mitigated against this issue as their non-swappable coins are forwarded to the burner module, clearing them from the iteration set each block, but there still can be spammed in the same way and will still need to iterate over all the denoms from that block.

Recommendation

We recommend adding the `txfeecollector` module account to `blockAccAddrs` to prevent arbitrary sends, and modifying `convertFeesAndSend` to forward non-swappable coins to the burner module regardless of the `sendSkipped` parameter.

Status: Resolved

2. Unbounded pending unlock iteration in epoch hook allows consensus halting Denial of Service attack

Severity: Critical

The rewards epoch hook returned by `GetUnlockPendingUnlockParticipantsHook` in `x/rewards/keeper/service_hooks.go:28-43` is registered in the `x/epochs` module hooks in `app/app.go:482`.

The `x/epochs` module executes the hook functions in the `BeginBlocker` path via the `AfterEpochEnd` function in `x/epochs/keeper/abci.go:44`. This places unlock processing in an unmetered consensus-critical execution path.

When triggered for the hourly expiration epoch, the hook calls `UnlockAllPendingUnlockParticipantsByEpoch` which fetches all pending unlock participants for the epoch using `GetAllEpochPendingUnlockParticipant` in `x/rewards/keeper/service_unlock.go:9`. That function performs an unbounded iteration over the full epoch bucket (`x/rewards/keeper/store_pending_unlock_participant.go:36-46`) and then processes each entry in sequence, executing a `x/bank` send and storage deletion per item.

An attacker can create many pending unlock entries by using multiple addresses that join and exit within the same hour. Since unlock keys are bucketed by target hour in `x/rewards/keeper/msg_server_staking_reward.go:484-486`, those entries are concentrated into one epoch and executed together in `BeginBlock` approximately 24 hours later (assuming the attacker uses a staking pool with the minimum lock time), resulting in `O(n)` asymptotic complexity in an unmetered hook, which can cause consensus timeouts or chain halts.

Recommendation

We recommend processing pending unlock participants in bounded batches with a strict maximum length per block, and carrying unprocessed entries forward to subsequent blocks until the epoch backlog is drained. This ensures per-block unlock work remains bounded even under adversarial queue growth.

Status: Resolved

3. Duplicate cancel orders allow order book Denial of Service attacks via processing batch exhaustion

Severity: Major

The `CancelOrder` handler in `x/tradebin/keeper/msg_server_order_book.go:151-183` allows multiple cancel

messages to be enqueued for the same order as long as the order exists and the creator is the owner. There is no guard against duplicate pending cancellations for a single order.

The `ProcessQueueMessages` function in `x/tradebin/keeper/queue_message_processor.go:81-125` runs from the `x/tradebin` module `EndBlocker` and processes up to `maxPerBlock` messages in sequence. The `processedCount` counter increments for every queued message, even when `cancelOrder` is a no-op because the order is already removed.

An attacker can submit a single transaction with many `MsgCancelOrder` messages for the same order and only pay the regular gas cost for each enqueue. When the number of cancel messages exceeds `maxPerBlock`, legitimate order messages from other users are deferred until all of the cancel orders have been processed.

Recommendation

We recommend tracking a per-order cancellation state and rejecting subsequent cancel requests for the same order before they are enqueued.

Status: Resolved

4. Lack of deduplication in `FillOrders` enables order book queue flooding

Severity: Major

In `x/tradebin/keeper/msg_server_order_book.go:201-247`, the `FillOrders` handler iterates over submitted `FillOrderItems` without performing any deduplication or validation to prevent repeated entries targeting the same price and amount.

A malicious actor can submit up to `50` identical `FillOrderItem` entries within a single transaction, each referencing the same order parameters. For every entry processed, the handler creates a separate `QueueMessage` (line `241`) and increments the global queue counter. This occurs without proportional dynamic gas costs tied to the number of queued messages.

As a result, an attacker can artificially inflate the order processing queue by duplicating fill instructions in a single transaction. Although this attack vector requires capital (as each fill must still be valid and funded), it enables inefficient state growth and queue congestion.

This behavior contributes to broader queue flooding risks, similar in nature to the "Duplicate cancel orders allow order book Denial of Service attacks via processing batch exhaustion" issue previously identified. If exploited at scale, it may degrade order book performance, increase block processing overhead, and negatively impact liveness for legitimate users.

Recommendation

We recommend deduplicating fill items by price within `ValidateBasic`, rejecting messages that contain repeated price entries.

Status: Resolved

5. Crossed book state allows griefing by blocking new sell orders

Severity: Major

The `CreateOrder` handler calls `checkPrice` in `x/tradebin/keeper/msg_server_order_book.go:61`, which returns early and without error when `GetAggregatedOrder` finds an opposite-side aggregate at the same price; skipping the `checkPriceInOrderBook` and `checkPriceInQueueMessages` better-price-exist guards.

The `ProcessQueueMessages` function in `x/tradebin/keeper/queue_message_processor.go:102` routes `CreateOrder` messages to the `addOrder` handler. The `addOrder` function only matches against an opposite aggregate at the same price via `fillAggregatedOrder`; if the aggregate order is filled and the remaining amount is greater than the dust amount, a resting order is saved at the same price.

This combination allows a crossed book to persist. An attacker can place a tiny sell at price `P` far from the best ask, then place a larger buy at the same price `P`. The buy consumes the tiny sell, and the remainder is posted by `addOrder` as a resting buy above the best ask, creating a crossed book. Once crossed, new sell orders priced below that buy are rejected by `checkPrice` because they are below the best bid, freezing sell-side liquidity until the crossed bid is cleared.

It may only be profitable for arbitrageurs to clear the crossed book if the combination of crossed order size and crossed spread is larger than the trading fees required to execute the trade.

Recommendation

We recommend preventing the crossed book state by removing the same-price short-circuit in `checkPrice`, so best-price guards always run, or by sweeping better-price aggregates before `addOrder` posts a remainder.

Status: Resolved

6. Unchecked staking reward duration addition can result in stranded funds

Severity: Major

The `UpdateStakingReward` function extends duration using unchecked arithmetic in `x/rewards/keeper/msg_server_staking_reward.go:142`:

```
stakingReward.Duration += uint32(durationInt)
```

If `durationInt` is large enough, this addition can wrap and reduce `stakingReward.Duration` below the current `stakingReward.Payouts` value.

This will trigger the distribution stop condition in `x/rewards/keeper/service_staking_reward.go:26`, where rewards are skipped when `sr.Payouts >= sr.Duration`. After a wrap, the reward can be treated as finished immediately, so scheduled distributions stop permanently.

The update path captures additional prize funds before writing the new duration (`x/rewards/keeper/msg_server_staking_reward.go:127` and `x/rewards/keeper/msg_server_staking_reward.go:137`), so a client-side duration mistake can lock newly captured funds in the module without a path for normal periodic distribution.

Additionally, the `UpdateStakingReward` function does not uphold the same max duration constraint as `CreateStakingReward`, where the validation of the `MsgCreateStakingReward` type ensures that the initial duration is less than `tenYearsInDays` in `x/rewards/types/message_create_staking_reward.go:67`.

Recommendation

We recommend using checked arithmetic before extending duration and return `ErrInvalidDuration` if the operation overflows or if the result is larger than `tenYearsInDays`

Status: Resolved

7. Unbounded reward hooks in epoch `BeginBlocker` enable Denial of Service

Severity: Major

Two hooks in the rewards module execute unbounded iterations within the epochs module's `BeginBlocker` path. These hooks are registered in `app/app.go:478–489`, dispatched through the epoch hook in `x/epochs/keeper/hooks.go:11–15`, and executed from

`x/epochs/keeper/abci.go13-58`. As a result, both run in an unmetered, consensus-critical execution path during BeginBlock.

The first hook, responsible for staking distribution (`x/rewards/keeper/service_hooks.go:11-26`), calls `DistributeAllStakingRewards`, which iterates over all staking reward entries using `IterateAllStakingRewards`. This operation is `O(n)` with respect to the number of concurrent staking reward records.

The second hook, responsible for expired pending trading reward cleanup (`x/rewards/keeper/service_hooks.go:45-60`), calls `removeExpiredPendingTradingRewards`. This function iterates over all expiration entries for the current epoch using a prefix iterator over `expireAt`. The complexity is `O(n)` in the number of rewards maturing during that epoch.

Both loops execute synchronously within `BeginBlock`, without explicit bounds or batching. Although governance-controlled fee parameters can increase the economic cost of creating large numbers of staking or trading rewards, this approach introduces an unnecessary tradeoff between usability and security. A sufficiently well-funded malicious actor can inflate the number of tracked reward entries or align many expirations within a single epoch, forcing validators to process large state iterations in one block.

Because this logic runs in a consensus-critical, unmetered path, excessive iteration can increase block processing time and potentially lead to timeouts or degraded liveness, constituting a denial-of-service (DoS) style availability risk.

Recommendation

We recommend processing both staking distributions and expired pending trading reward removals in bounded batches, enforcing a strict maximum number of entries processed per block. Any remaining items should be deferred and carried forward to subsequent blocks until the backlog is fully drained.

Status: Resolved

8. Trading rewards only validate order book markets, excluding AMM pools

Severity: Major

In `x/rewards/keeper/msg_server_trading_reward.go:44`, the `CreateTradingReward` function validates the target market by invoking `k.tradeKeeper.MarketExists(ctx, tradingReward.MarketId)`. However, the `MarketExists` implementation only verifies the existence of order book markets and does not recognize AMM liquidity pools, which use a distinct identifier format generated via `pool.GetId`.

Both order book trades and AMM swaps trigger the `GetOnOrderFillHook` hook, which distributes trading rewards. This indicates that AMM swaps are intended to participate in the trading rewards mechanism. Despite this, users are currently unable to create trading rewards for AMM liquidity pools because the market validation step rejects AMM pool IDs as invalid.

This creates an inconsistency between reward distribution logic and reward creation validation.

Recommendation

We recommend updating the market validation logic to also check for AMM liquidity pools using the pool ID format. The validation should accept both order book market IDs and AMM pool IDs as valid trading reward targets.

Status: Resolved

9. Rounding-induced lockup risk in `claimPending` enabling temporary denial of exit

Severity: Major

In `x/rewards/keeper/msg_server_staking_reward.go:442-481`, the `claimPending` function calculates pending staking rewards using decimal arithmetic and truncates the result to an integer before distribution.

The function only treats the condition `DistributedStake == JoinedAt` as "no rewards," returning a zero payout in that case.

However, when `DistributedStake != JoinedAt`, the reward is computed as: `deposited * (S_now - S_joined)`.

The result is truncated to an integer value before validation. If the computed decimal reward is greater than zero but less than one unit of the reward denomination (i.e., 0 < reward < 1), truncation reduces it to zero. The subsequent validation requires the reward to be strictly positive, and if the truncated value equals zero, the function returns an error instead of allowing a zero payout.

For small participants in large staking pools, or during early distribution phases, this condition is realistic and not edge-case behavior.

As a result:

- `ClaimStakingRewards` may revert.
- `JoinStaking` (for existing participants) may revert.
- `ExitStaking`, which unconditionally invokes `claimPending` and aborts on error, may revert.

This creates a temporary lockup condition where users are unable to exit staking or claim rewards until additional distributions accumulate enough to push their truncated reward above one unit. There are no parameter safeguards in the current configuration that prevent this rounding-induced state.

Although funds are not permanently lost, this behavior can effectively result in a denial of service against small participants, preventing them from exiting the staking program due to arithmetic truncation effects.

Recommendation

We recommend modifying `claimPending` to treat truncated zero rewards as a valid zero-claim scenario rather than an error condition.

Status: Resolved

10. Fee denomination context not set during `ReCheckTx` causing mempool evictions for non-native fees

Severity: Major

In `x/txfeecollector/ante/check_fee_denom.go:25-30`, the `ValidateTxFeeDenomsDecorator` immediately short-circuits when `ctx.IsReCheckTx` is `true` and forwards execution to the next ante handler without executing its validation logic.

This decorator is responsible not only for validating that a transaction contains a single fee denomination, but also for storing that denomination in the context under `FeeDenomKey`.

The `DeductFeeDecorator` later relies on this context value in `getContextMinGasPrices` to compute the required minimum gas price in the correct denomination. This includes support for non-native fee tokens, which are priced against the native coin using `TradeKeeper.GetDenomSpotPriceInNativeCoin`.

Because the decorator does not execute on `ReCheckTx`, it does not set `FeeDenomKey` in the context during that phase. Consequently, `getContextMinGasPrices` observes an empty fee denomination and falls back to the protocol's native denom defined in params. It then constructs a minimum-fee requirement in the native denomination, while the transaction's actual fee may be denominated in a non-native token.

Since the SDK fee comparison logic requires matching denominations, transactions that originally passed `CheckTx` with valid non-native fees may now appear to have insufficient fees during `ReCheckTx`. On nodes with mempool recheck enabled, such transactions are evicted from the mempool.

Although this behavior does not affect `DeliverTx` execution or consensus safety, it effectively breaks multi-denomination fee support at the mempool level. This degrades

mempool liveness and negatively impacts user experience for transactions paying fees in non-native tokens.

Recommendation

We recommend ensuring that `ValidateTxFeeDenomsDecorator` executes its fee-denomination extraction and context-setting logic during `ReCheckTx`, or otherwise guaranteeing that `FeeDenomKey` is consistently populated in the context before fee comparison occurs.

Status: Resolved

11. Validation allows the creation of a liquidity pool with one empty asset

Severity: Minor

In `x/tradebin/types/message_create_liquidity_pool.go:33`, the `ValidateBasic` function checks if `len(msg.Base) == 0 && len(msg.Quote) == 0` using a logical `AND` operator. This means the validation only fails if both assets are empty.

Consequently, if only one asset is empty, the validation passes, allowing the creation of a liquidity pool with only one asset defined.

A liquidity pool requires both a base and a quote asset to function correctly. The validation should fail if either asset is missing.

Recommendation

We recommend changing the logical operator from `&&` to `||` on `line 33`, so the validation fails if either `msg.Base` or `msg.Quote` is empty.

Status: Resolved

12. Takers of queued opposite-side liquidity can pay the maker fee rate

Severity: Minor

The fee role for `MsgCreateOrder` is decided in `captureMsgFees` in `x/tradebin/keeper/msg_server_order_book.go:327-332`, which sets `isTaker` only from `GetAggregatedOrder` at the submitted price for the opposite side in line `329`. This check does not include opposite-side orders already accepted into the queue but not yet processed into the aggregated book state.

Order intake and execution are decoupled. `CreateOrder` captures fees before enqueue in `x/tradebin/keeper/msg_server_order_book.go:58-149`, then stores the message via `SetQueueMessage` in `x/tradebin/keeper/store_queue_message.go:19-32`.

Queue processing occurs later in `EndBlock` via `ProcessQueueMessages` in `x/tradebin/module/module.go:72-84`. Messages are iterated in key order in `IterateAllQueueMessages` in `x/tradebin/keeper/store_queue_message.go:56-68`; during processing, an earlier opposite-side message can be added to the book in `addOrder` in `x/tradebin/keeper/queue_message_processor.go:225-282`, and a later same-price order can then execute against that liquidity as taker in line `240` and line `255`. The pre-enqueue price check in `checkPriceInQueueMessages` in `x/tradebin/keeper/msg_server_order_book.go:474-522` does not force taker classification for equal-price queued liquidity, so this path remains reachable.

Resulting taker executions can be charged at maker rates when opposite-side liquidity is present in queue timing but absent from aggregated state at submission, causing fee-policy drift that reduces taker fee capture and weakens maker incentives; this is particularly exploitable by MEV actors who can order transactions to snipe stale quotes while still paying maker fees.

Recommendation

We recommend determining maker/taker role from an execution-consistent view that includes relevant queued opposite-side messages, or moving fee assessment to execution time so charged role matches realized maker/taker behavior.

Status: Acknowledged

13. Inconsistent zero-reward handling enables no-op claim-event spam

Severity: Minor

The `ClaimStakingRewards` handler in `x/rewards/keeper/msg_server_staking_reward.go:318-353` always follows the success path after `claimPending` in line `333`, including participant persistence in line `338` and `StakingRewardClaimEvent` emission in line `340`.

In `claimPending` in lines `442-481`, zero-reward outcomes are handled inconsistently. When `distributedStake` equals `joinedAt`, the function returns a zero coin and no error in lines `457-460`.

However, the latter guard returns an error for non-positive computed rewards in lines `463–465`. This inconsistency allows a no-reward claim to be treated as successful and emitted with a zero `Amount` in line `344`.

Participants can repeatedly submit no-op claims to generate successful claim events without payout, which can mislead off-chain monitoring and add avoidable event load.

Recommendation

We recommend unifying zero-reward handling in `claimPending` by returning an explicit error when there are no claimable rewards.

Status: Resolved

14. Delimiter-based key encoding allows cross-market prefix pollution

Severity: Minor

The `x/tradebin` module builds market and order keys by concatenating raw components with `'/'`.

At the same time, `x/tokenfactory` permits user-controlled subdenoms containing '/', `MsgCreateDenom.ValidateBasic` only rejects '_', denoms are built as `factory/{creator}/{subdenom}`, and denom parsing explicitly supports slash-bearing subdenoms.

Because market identifiers are reused inside order and aggregate key paths, a user can create market IDs that are prefix extensions of other market paths and contaminate market-scoped prefix scans.

Short collision sequence:

1. Attacker creates two quote denoms in `x/tokenfactory`: `Q1 = factory/<creator>/q` and `Q2 = factory/<creator>/q/sell`.
2. Attacker creates markets in `x/tradebin` with the same base denom `B`: `M1 = B/Q1` and `M2 = B/Q2`.
3. This gives a suffix relationship: `M2 = M1 + "/sell"`.
4. When `x/tradebin` builds aggregated-order prefixes for `(M1, sell)`, the prefix overlaps the keyspace of market `M2`, so scans for `M1` can include entries from `M2`.

As market and order lookups in `x/tradebin` rely on these raw prefixes, crafted markets can pollute scans intended for another market. In practice, this can cause cross-market orderbook contamination and false "better price exists" or "price outdated" checks during order placement.

We classify this as minor because the attacker can only influence markets they create or can fund with a controllable denom structure, and the impact is primarily integrity and availability degradation (order rejection and misleading views) rather than direct fund theft.

Recommendation

We recommend encoding each key segment safely so prefixes cannot collide. For example, concatenating length-prefixed byte representations of the key components.

Status: Resolved

15. Trading reward expiration uses a hardcoded period instead of the user-defined duration field

Severity: Minor

In `x/rewards/keeper/keeper.go:107`, the `getNewTradingRewardExpireAt` function calculates the expiration timestamp by adding the `expirationPeriodInHours` constant (720 hours, or 30 days) to the current epoch count, ignoring the `tradingReward.Duration` field entirely.

This is problematic because the `MsgCreateTradingReward` message in `x/rewards/types/message_create_trading_reward.go` accepts and validates a `Duration` field, and this value is stored in the `TradingReward` struct. Users who specify a custom duration expect the trading reward to expire accordingly, but the system always applies a fixed 30-day period regardless of the configured value.

Consequently, all trading rewards expire exactly 30 days after the start date, regardless of the user-specified duration.

Recommendation

We recommend modifying `getNewTradingRewardExpireAt` to use `tradingReward.Duration` instead of the hardcoded `expirationPeriodInHours` constant when calculating the expiration timestamp for active rewards.

Status: Resolved

16. Multiple pending trading reward activations for the same market lead to orphaned rewards

Severity: Minor

In `x/rewards/keeper/msg_server_gov.go:52`, the `ActivateTradingReward` function calls `SetMarketIdRewardId` to map a market ID to the newly activated reward ID. The function checks whether the reward itself is already active in line `25` using `GetActiveTradingReward`, but does not validate whether another reward is already active for the same market.

The issue is that governance can activate multiple pending trading rewards for the same markets. The second activation overwrites the `MarketIdRewardId` mapping, causing the `GetOnOrderFillHook` to track trades only for the second reward.

The previous reward remains in the active store with its own expiration, but its leaderboard will no longer receive updates. When the reward expires, `distributeTradingRewards` attempts to pay winners from a stale or empty leaderboard, potentially leaving captured funds undistributed in the module.

Consequently, the orphaned reward either distributes prizes based on an outdated leaderboard or fails to distribute entirely if the leaderboard is empty.

Recommendation

We recommend checking `GetMarketIdRewardId` for the target market during `ActivateTradingReward` and rejecting the activation if an active reward already exists for that market.

Status: Resolved

17. Missing duration cap during staking reward updates allows bypass of the maximum duration constraint

Severity: Minor

In `x/rewards/keeper/msg_server_staking_reward.go:142`, the `UpdateStakingReward` function adds the requested `durationInt` to the existing `stakingReward.Duration` without checking whether the resulting total exceeds the `maxAllowedDuration` constant defined in `x/rewards/types/message_create_trading_reward.go:16`.

This is problematic because staking reward durations are not allowed to exceed the `maxAllowedDuration` constant, as enforced by the `CreateStakingReward` entry point. By repeatedly calling `UpdateStakingReward` with incremental durations, a reward creator could accumulate a total duration that exceeds this intended cap.

Recommendation

We recommend adding a validation in `UpdateStakingReward` to ensure the total `stakingReward.Duration` after the update does not exceed `maxAllowedDuration`.

Status: Resolved

18. Break statement in order filling skips potentially fillable orders at the same price level

Severity: Minor

In `x/tradebin/keeper/queue_message_processor.go:628-630`, the `fillAggregatedOrder` function iterates over opposite orders at the same price level. When `getExecutedAmount` returns zero for a given order, the function executes a `break` statement, terminating the entire iteration over remaining orders.

This is problematic because the remaining orders will not be fulfilled, reducing market efficiency. Subsequent orders with a larger amount at the same price could satisfy the fill constraints. For example, with a `minAmount` of 10 and a `msgAmount` of 15, an order with an `orderAmount` of 18 returns zero due to remainder constraints, while a later order with an `orderAmount` of 100 would successfully fill.

Consequently, the `break` results in the message owner receiving an unnecessary refund and leaves maker orders unfilled.

Recommendation

We recommend replacing the `break` statement with `continue` so that the iteration continues evaluating remaining orders at the same price level, allowing all compatible orders to be considered for filling.

Status: Resolved

19. Pending trading reward removal commits partial state when burn operation fails

Severity: Minor

In `x/rewards/keeper/service_trading_reward.go:16-22`, the `removeExpiredPendingTradingRewards` function calls `RemovePendingTradingRewardExpiration` and `RemovePendingTradingReward` before attempting to burn the captured coins via `BurnCoins` in line `31`. If the burn fails, the function logs the error and continues to the next item.

However, the enclosing epoch hook in `x/rewards/keeper/service_hooks.go:58` always returns `nil`, causing `ApplyFuncIfNoError` to commit all state changes. Pending reward records are permanently deleted, while the associated coins remain in the module account. Since the records no longer exist, there is no reference to identify or recover these orphaned funds.

Consequently, the captured prize funds are permanently locked in the module account, with no mechanism for recovery or redistribution, resulting in a direct loss of user-deposited funds.

Recommendation

We recommend wrapping each expired reward iteration in its own `ApplyFuncIfNoError` call and returning errors from the burn operation, so that a failed burn fully rolls back the record deletions for that individual reward.

Status: Resolved

20. `JoinStaking` allows joining expired or finished rewards

Severity: Minor

In `x/rewards/keeper/msg_server_staking_reward.go:156-173`, the `JoinStaking` handler does not verify whether the staking reward has already completed its distribution lifecycle before accepting new participants.

As a result, users can stake tokens into a reward program that has already finished distributing all payouts. In such cases, the staked tokens are transferred to and locked within the module account, but the participant receives no rewards because no further distributions will occur.

This behavior creates a funds lockup risk and a significant user experience issue. Although no funds are directly lost, users may unknowingly deposit tokens into an inactive reward program and receive zero return, with capital effectively immobilized until an exit path is triggered (if allowed).

The absence of a completion-state validation introduces an economic correctness flaw and increases the risk of user harm through misconfiguration or interface misunderstandings.

Recommendation

We recommend adding a check at the beginning of JoinStaking to verify that `stakingReward.Payouts < stakingReward.Duration` before allowing new participants to join. This prevents users from inadvertently locking tokens in a reward that will never pay out.

Status: Acknowledged

21. Global queue counter never resets while failed messages persist

Severity: Minor

In the `ProcessQueueMessages` function in `x/tradebin/keeper/queue_message_processor.go:118-124`, the queue message counter only resets when the queue is completely empty.

Failed messages are intentionally retained for retry (lines 150–153), but if a message permanently fails, the counter never resets.

Since the counter is global across all markets in `store_counter.go:9` and increments with every new order `store_queue_message.go:31`, the dynamic gas surcharge in `msg_server_order_book.go:96` grows over time, eventually making order creation prohibitively expensive or blocked on all markets.

Recommendation

We recommend basing the gas surcharge on actual pending queue depth rather than the historical message-id counter.

Status: Acknowledged

22. Genesis validation only checks params, allowing malformed state import

Severity: Minor

In `x/tradebin/types/genesis.go:20-24` and `x/rewards/types/genesis.go:20-24`, the `Validate` function only validates the `Params` field of the genesis state. Other fields are not validated before being imported by `InitGenesis`.

A malformed genesis file could inject invalid orders with corrupt prices, negative amounts, invalid market IDs, or other invalid state that would be rejected during normal runtime. This corrupted state could cause panics, incorrect order matching, or other system failures.

Recommendation

We recommend adding comprehensive validation in the `Validate` function to check all genesis state fields, including orders, liquidity pools, and queue messages. Each item should be validated using the same rules applied during runtime message handling.

Status: Acknowledged

23. Time-of-check-time-of-use gap in deep liquidity validation allows overpayment

Severity: Minor

In `x/tradebin/keeper/service_fee_payer.go:34-40`, the `CaptureAndSwapUserFee` function retrieves a liquidity pool without re-checking deep liquidity. The pool's deep liquidity status is validated earlier in the transaction flow by the `ValidateTxFeeDenomsDecorator` ante handler in `x/txfeecollector/ante/check_fee_denom.go:64`. However, between the ante handler check and this function

call, the previous swap in the same transaction could have drained the pool's native token reserves.

This creates a time-of-check-time-of-use issue where a pool that had deep liquidity during validation may no longer have it when the swap is executed. This could result in users paying a large amount of their preferred fee token for a small amount of native tokens due to poor pool liquidity.

Recommendation

We recommend adding a deep liquidity check before retrieving the pool on line `35`, or before executing the swap on line `64`. If deep liquidity is no longer present, fall back to capturing fees in the native denomination.

Additionally, we recommend introducing an upper slippage bound parameter (e.g. `MaximumSlippageForModuleSwap`) for preferred-denom fee conversion and failing before capture when `requiredInput` exceeds the bound.

Status: Resolved

24. Function returns partial swap results on error without context flush

Severity: Minor

In `x/tradebin/keeper/service_denom.go:159,174`, when an error occurs during multi-coin swapping, the function returns `swapResult, err` instead of `nil, err`. If one coin was successfully swapped and added to `swapResult` before the error occurred, the caller receives a non-empty `swapResult` even though the swap was not flushed to the context state.

This creates a misleading return value where `swapResult` indicates coins were swapped, but those swaps never actually occurred in the blockchain state. While the current caller ignores `swapResult` when `err != nil`, this API design is error-prone and could lead to bugs if other callers are added.

Recommendation

We recommend changing all error return statements to `return nil, err` instead of `return swapResult, err` to clearly indicate that no swaps were successfully committed when an error occurs.

Status: Resolved

25. Trading reward distribution failures are silently ignored

Severity: Minor

In `x/rewards/keeper/service_trading_reward.go:92`, when the `SendCoinsFromModuleToAccount` call fails during trading reward distribution, the error is silently ignored without logging. This means reward distribution failures go unnoticed, potentially causing users to lose rewards without any indication of what went wrong.

Silent failures make debugging difficult and can erode user trust if rewards are frequently missed without explanation.

Recommendation

We recommend detecting the send failure and logging it. Consider whether distribution failures should be treated as critical errors that prevent the transaction from succeeding.

Status: Resolved

26. Burning IBC voucher tokens can lead to accounting discrepancies

Severity: Minor

In `x/rewards/keeper/service_trading_reward.go:31`, expired pending trading rewards are burned via the `BurnCoins` function.

However, the `CreateTradingReward` function in `x/rewards/keeper/msg_server_trading_reward.go:38-42` only validates that the prize denom has supply, without restricting IBC voucher tokens. This allows users to create trading rewards using IBC denoms (e.g., `ibc/27394FB...`).

When a trading reward with an IBC token prize expires before activation, the `removeExpiredPendingTradingRewards` function burns the IBC vouchers from the rewards module. Burning these vouchers creates a supply mismatch between the source chain (where tokens remain locked) and the destination chain (where vouchers are destroyed), violating IBC accounting invariants. This results in tokens being permanently locked on the source chain with no corresponding vouchers to redeem them.

Recommendation

We recommend either swapping the expired IBC tokens to the native denom before burning (similar to how fees are handled), or sending the expired tokens to the burner module for proper disposal.

Status: Resolved

27. Weak error handling in `CalculateMinAmount` may bypass minimum order protections

Severity: Minor

In `x/tradebin/keeper/service_order_book.go:12-36`, the `CalculateMinAmount` helper derives a minimum order amount from a provided price string. The function converts the price into a decimal, computes the reciprocal, rounds up to avoid dust, multiplies the result by two as a buffer, and returns the truncated integer value.

However, its error handling is fragile and potentially misleading. If parsing the price string fails, the function logs the error using `fmt.Println` and returns zero as the minimum amount. Similarly, if the parsed price equals zero, the function silently returns zero.

Recommendation

We recommend returning an explicit error instead of silently defaulting to zero when price parsing fails or when the price is invalid.

Status: Resolved

28. Deprecated `x/crisis` module contains unpatched vulnerabilities with no upstream fix

Severity: Informational

In `cmd/bzed/cmd/commands.go:22-58`, the application imports and initializes the `x/crisis` module from `github.com/cosmos/cosmos-sdk@v0.50.15`.

This module is affected by two known vulnerabilities with no upstream fix available:

- GO-2023-1881 documents that the `x/crisis` module does not charge the configured `ConstantFee` when processing invariant check transactions, allowing any user to trigger potentially expensive invariant checks without paying the intended fee.
- GO-2023-1821 documents that the `x/crisis` module does not cause a chain halt when an invariant is violated, thereby defeating the module's primary purpose.

Since both vulnerabilities lack upstream fixes and the module is effectively deprecated in newer Cosmos SDK versions, it provides no meaningful safety guarantees.

Recommendation

We recommend removing the `x/crisis` module from the application entirely.

Status: Resolved

29. Incorrect amount tracked in liquidity addition event emits misleading data

Severity: Informational

In `x/tradebin/keeper/service_denom.go:360`, the `ModuleAddLiquidityWithNativeDenom` function tracks `addedCoins` using `optimalQuote`. The variables `optimalBase` and `optimalQuote` represent the calculated optimal base and quote amounts, respectively.

The issue is that if the input `coin.Denom` is the pool base asset, the function should track `optimalBase` as the added coin amount rather than `optimalQuote`, since `optimalBase` reflects the actual amount of the input denomination contributed to the pool. Using `optimalQuote` causes the emitted event data to reference the incorrect denomination amount, leading to misleading information in logs and events.

Recommendation

We recommend changing the tracked `addedCoins` to use `optimalBase` when `coin.Denom` matches the pool base asset, ensuring the event data accurately reflects the contributed amount.

Status: Resolved

30. Pervasive use of `string` types for numeric and coin parameters bypasses Go type safety across all modules

Severity: Informational

Across the codebase, protobuf message definitions use `string` for fields that semantically represent integers, unsigned integers, decimals, or coins. This forces every consumer to manually parse and validate the string at runtime, forgoing compile-time type safety.

For example, the `MsgCreateStakingReward` in `proto/bze/rewards/tx.proto:47~57` defines `string` for the `prize_amount`, `duration`, `min_stake`, and `lock` fields, requiring them to be parsed via `math.NewIntFromString` and `strconv.Atoi` in `x/rewards/types/message_create_staking_reward.go:39-80`. If these were defined as their dedicated types, the parsing overload would be unnecessary.

Consequently, these introduce many manual parse-and-validate call sites, each of which creates opportunities for inconsistent validation or conversion errors that could be caught at compile time by implementing proper types.

Recommendation

We recommend migrating numeric string fields to their appropriate protobuf types. The following are examples:

- `uint32`/`uint64` for durations, slots, and counts.
- `string` with `(gogoproto.customtype) = "cosmossdk.io/math.Int"` for integer amounts.
- `string` with `(gogoproto.customtype) = "cosmossdk.io/math.LegacyDec"` for decimal values like prices and ratios.
- `cosmos.base.v1beta1.Coin` for coin amounts.

Status: Acknowledged

31. Duplicate validation in multi-swap

Severity: Informational

In `x/tradebin/types/message_multi_swap.go:48`, the `ValidateBasic` method of `MsgMultiSwap` performs an `IsPositive` check on the swap input and minimum output amounts (`msg.Input` and `msg.MinOutput`). This check is redundant because validation is already performed earlier in lines `40-46`.

While not a security vulnerability, dead code in validation logic can obscure the actual validation flow and introduce maintenance burdens.

Recommendation

We recommend removing the redundant `IsPositive` check to improve code clarity and reduce unnecessary validation logic.

Status: Resolved

32. Updating `ValidatorMinGasFee` to a non-native denomination breaks the fee validation logic

Severity: Informational

In `x/txfeecollector/ante/validator_tx_fee.go:78-80`, the `checkTxFeeWithValidatorMinGasPrices` function calculates the required fee for non-native denominations by dividing `localNativeGasPrice` by `txDenomPrice.Amount`, which assumes `ValidatorMinGasFee.Denom` is always the native coin.

However, `MsgUpdateParams` in `x/txfeecollector/keeper/msg_update_params.go:12-28` allows governance to update `ValidatorMinGasFee` to any denomination, as the `validateValidatorMinGasFee` function in `x/txfeecollector/types/params.go:39` only checks that the denomination is non-empty and the amount is non-negative.

If governance sets `ValidatorMinGasFee.Denom` to a non-native coin, the division in `x/txfeecollector/ante/validator_tx_fee.go:80` becomes semantically incorrect because `localNativeGasPrice` would represent the non-native denomination minimum gas price while `txDenomPrice.Amount` represents the spot price in the native denomination.

Consequently, the fee validation would produce incorrect required fee amounts, potentially allowing transactions with insufficient fees or rejecting transactions with valid fees.

We classify this issue as informational severity because governance is not expected to update the denom to a non-native coin.

Recommendation

We recommend adding a validation in the `validateValidatorMinGasFee` function to ensure that `ValidatorMinGasFee.Denom` matches the expected native denomination, or adjusts the fee calculation logic to handle arbitrary denominations correctly.

Status: Resolved

33. Misleading variable naming in `MultiSwap` swap loop

Severity: Informational

In the `MultiSwap` function in `x/tradebin/keeper/msg_server_amm.go:303-316`, the variable `outputCoin` is initialized with the user's input coin before any swap has occurred.

It then serves a dual role as the output of the previous hop and input to the next hop, but the name only reflects one of those roles.

Additionally, in line `312` it is passed as the input parameter to `onSwapSuccess`, further confusing which coin represents input vs output in the swap event context. This reduces readability and increases the risk of future maintainers misunderstanding the swap loop's data flow.

Recommendation

We recommend renaming `outputCoin` to something that reflects its role, such as `inputCoin`.

Status: Resolved

34. Unused `ErrSample` error variable

Severity: Informational

In `x/txfeecollector/types/errors.go:12`, the `ErrSample` error is defined as `sdkerrors.Register(ModuleName, 1101, "sample error")`.

This error is a scaffold placeholder that is never referenced anywhere in the codebase. Unused error registrations occupy error code space and can cause confusion for developers maintaining the module.

Recommendation

We recommend removing the `ErrSample` variable and its associated error code registration to clean up the codebase.

Status: Resolved

35. Silent error handling in `transformPrice` enables state corruption via malformed genesis import

Severity: Informational

In `x/tradebin/types/key_order.go:76-78`, the `transformPrice` function silently returns the unparsed price string when decimal parsing fails.

While runtime order creation validates prices through `ValidateBasic`, the `InitGenesis` function in `x/tradebin/module/genesis.go:25-27` calls `SaveOrder` directly without validation, allowing malformed genesis files to inject invalid price strings into the store.

This can corrupt the price-based ordering system used by the queue processor for price-time priority matching in `keeper/queue_message_processor.go:609`.

Recommendation

To prevent state corruption, we recommend adding validation to `InitGenesis` before calling `SaveOrder` to ensure genesis imports cannot bypass price validation.

Additionally, we recommend modifying `transformPrice` to handle parse errors properly rather than silently returning the unparsed string. The function should either panic with a descriptive error (since it is called in key generation paths where errors cannot be propagated), or the calling code should validate prices before passing them to `transformPrice`.

Status: Resolved

36. Misleading error messages

Severity: Informational

In `x/tradebin/keeper/msg_server_order_book.go:431`, the error message states *"could not get buy orders pagination query"* when the code is searching for sell orders (line `428`). The error message should state "could not get sell orders pagination query" to accurately reflect the operation being performed.

Furthermore, in `app/genesis_account.go:33`, the error message states that vesting start time cannot occur "before" the end time.

Recommendation

We recommend changing the error message on line `431` to "could not get sell orders pagination query" to match the actual order type being queried.

We also recommend correcting the error message on line `33` to state that the vesting start time cannot be "at or after" the end time.

Status: Resolved

37. Format string missing denom parameter in error message

Severity: Informational

In `x/txfeecollector/ante/check_fee_denom.go:57-60`, the error message uses a format string *"invalid fee supplied. can not use %s denom as tx fee"* but does not provide the denom value as a parameter to `sdkerrors.Wrap`.

This results in the literal string `"%s"` being displayed in the error message instead of the actual denom value, reducing the usefulness of the error for debugging.

Recommendation

We recommend using `sdkerrors.Wrapf` instead of `sdkerrors.Wrap` and providing the denom value as a parameter, or restructuring the error message to not require formatting.

Status: Resolved

38. Magic string used for context key instead of a defined constant

Severity: Informational

In `x/tradebin/keeper/service_fee_payer.go:103`, the `getCtxDenom` function retrieves a value from the context using the magic string `"fee_denom"`.

However, the constant `FeeDenomKey` is defined in `x/txfeecollector/ante/check_fee_denom.go:11` and used on line `73` of that file when setting the context value. Using the magic string instead of the constant creates a maintenance risk if the key name ever changes.

Recommendation

We recommend importing and using the `FeeDenomKey` constant instead of the magic string `"fee_denom"` to ensure consistency between setting and retrieving the context value.

Status: Resolved

39. Missing defensive validation for negative `amountTraded` in hook function

Severity: Informational

In `x/rewards/keeper/service_hooks.go:112`, the `GetOnOrderFillHook` function converts `amountTraded` to an integer without checking if it is negative.

While the current code paths ensure `amountTraded` cannot be negative (from `getExecutedAmount` for order book fills or `swapTokens` outputs for AMM swaps), this function is called from a hook interface.

Future code changes could introduce new callers that pass negative values, leading to unexpected behavior.

Recommendation

To improve defensive programming, we recommend adding a validation check to ensure `amountTraded` is not negative before processing. This prevents potential issues if new code paths are added that call this hook with invalid values.

Status: Resolved

40. Missing validation for negative reserves could allow state corruption

Severity: Informational

In `x/tradebin/keeper/service_amm.go:52-55`, the `BalanceProvidedAmounts` function checks if reserves are zero (`if reserveBase.IsZero() || reserveQuote.IsZero()`), but does not validate that reserves are positive. While the current code paths ensure reserves cannot be negative, this defensive check is missing at the function boundary.

If future code changes bypass the upstream validation, negative reserves could corrupt the liquidity pool state and cause incorrect swap calculations.

Recommendation

We recommend changing the validation on line 53 to `!reserveBase.IsPositive() || !reserveQuote.IsPositive()` to explicitly check that both reserves are positive, even though current logic prevents negative values.

Status: Resolved

41. Misleading comment may cause confusion

Severity: Informational

In `x/tradebin/keeper/msg_server_amm.go:423-426`, there is a misleading comment reading that it *"checks that the difference is not greater than 1e-6"*, while there is no difference tolerance logic applied, the fees must add up to exactly 1.

Recommendation

We recommend removing the comment.

Status: Resolved

42. Variable assignment is immediately overwritten

Severity: Informational

In `x/rewards/keeper/msg_server_staking_reward.go:176`, a variable is assigned a value that is immediately overwritten on line `177` before being used. This serves no purpose and adds unnecessary code complexity.

Recommendation

We recommend removing the redundant assignment to improve code clarity.

Status: Resolved

43. Invalid creator address errors are mislabeled as unauthorized

Severity: Informational

In CreateLiquidityPool in `x/tradebin/keeper/msg_server_amm.go:23-25`, a Bech32 parse failure from `sdk.AccAddressFromBech32` is wrapped with `sdkerrors.ErrUnauthorized` instead of `sdkerrors.ErrInvalidAddress`.

The same pattern appears in `RemoveLiquidity` in lines `126-128`, `AddLiquidity` in lines `203-205`, and `MultiSwap` in lines `281-283`.

This mislabeling can cause incorrect diagnostics for clients and monitoring systems that rely on SDK error kinds.

Recommendation

We recommend returning `sdkerrors.ErrInvalidAddress` for `sdk.AccAddressFromBech32` failures in the affected handlers in `x/tradebin/keeper/msg_server_amm.go`.

Status: Resolved

44. Pending unlock accumulation logic runs unnecessarily for zero-lock rewards

Severity: Informational

In `x/rewards/keeper/msg_server_staking_reward.go:494-508`, the `beginUnlock` function retrieves any existing pending unlock and accumulates amounts (lines `494-501`) before checking if the lock period is zero (line `505`). When lock is zero, `performUnlock` sends the accumulated amount and returns, but the original pending unlock remains in storage with the unaccumulated value.

Currently, this is not exploitable because staking reward lock periods are immutable after creation. If a reward has lock 0, it has always been 0, so no pending unlock exists to accumulate.

However, if lock periods become mutable in the future, a user could participate when lock is X (creating a pending unlock with amount A), then participate again after lock changes to 0 (amount B). The code would accumulate to A+B, send it via `performUnlock`, but leave the original pending unlock with amount A in storage.

When `UnlockAllPendingUnlockParticipantsByEpoch` later processes that epoch, amount A would be unlocked again, causing a double spend.

Recommendation

We recommend moving the lock period check to the beginning of the `beginUnlock` function. If `sr.Lock == 0`, call `performUnlock` immediately with the new amount and return, bypassing the accumulation logic entirely. This prevents any interaction with existing pending unlocks when an immediate unlock is intended.

Status: Resolved

45. Invalid amount comparison silently uses undefined behavior

Severity: Informational

In `x/rewards/keeper/service_hooks.go:163-164`, when comparing trading reward amounts to determine priority, an error during amount parsing is silently ignored.

If either amount string is invalid and cannot be parsed, the comparison result is undefined. This should not happen under normal conditions, but if it does occur due to state corruption, the silent error could cause incorrect reward priority ordering.

Recommendation

We recommend detecting the parsing error and either treating the invalid amount as lower priority or logging the error with a clear indication of which amount failed to parse.

Status: Resolved

46. Multi-swap route length error message incorrectly implies that zero routes are valid

Severity: Informational

In `ValidateBasic` in `x/tradebin/types/message_multi_swap.go:28-29`, the condition rejects route lengths `<=0` and `>MaxRoutes`, but the returned message says *"routes length must be between 0 and %d"*. This wording incorrectly suggests that 0 routes are valid.

The validation logic is correct, but the error string does not match it and can mislead users and integrators, creating avoidable integration confusion.

Recommendation

We recommend updating the error string in `x/tradebin/types/message_multi_swap.go:29` to state that route length must be between 1 and `MaxRoutes`.

Status: Resolved

47. Missing validation for distribution amount

Severity: Informational

In `x/rewards/types/message_distribute_staking_rewards.go:19-25`, the `ValidateBasic` function does not validate the `amount` field.

While downstream handlers may perform validation, it is best practice to validate all message fields in `ValidateBasic` to catch invalid inputs early and provide clear error messages to users.

Recommendation

We recommend adding amount validation in `ValidateBasic` to ensure the amount is positive and not zero before processing the distribution message.

Status: Resolved

48. Staking reward duration parsing is inconsistent between validation and execution

Severity: Informational

In `ValidateBasic` in `x/rewards/types/message_update_staking_reward.go:26`, `msg.Duration` is parsed with `strconv.ParseUint`, while `UpdateStakingReward` in `x/rewards/keeper/msg_server_staking_reward.go:113` parses the same field with `strconv.ParseInt`.

This allows values in the `uint32`-only range to pass message validation but fail later in the handler with a parse error.

The mismatch creates inconsistent request behavior and confusing failure timing for clients and integrations, because a message accepted by basic validation can still be rejected during execution.

Recommendation

We recommend using the same numeric parser and range in both `ValidateBasic` and `UpdateStakingReward` for `msg.Duration` so validation and execution enforce identical constraints.

Status: Resolved

49. Missing `nil` check on optional `TradingKeeper` dependency leading to potential runtime panic

Severity: Informational

In `x/rewards/keeper/msg_server_staking_reward.go:63-67`, the `MsgCreateStakingReward` handler conditionally processes a fee through the `TradingKeeper` dependency.

While the module's dependency injection configuration explicitly marks `TradeKeeper` as optional (`optional:"true"`), the implementation assumes that the dependency is always initialized when `CreateStakingRewardFee` is positive and invokes `CaptureAndSwapUserFee` without verifying that `k.tradeKeeper` is non `nil`.

If the module is deployed in an environment where `TradeKeeper` is not wired and fee handling is enabled, this assumption may result in a `nil` pointer dereference at runtime. Such a condition would trigger a panic, potentially leading to a denial of service scenario for transactions invoking this message type.

Although this does not constitute an exploitable vulnerability in the current deployment, it represents an implicit configuration assumption and introduces a portability and maintainability risk.

Recommendation

We recommend explicitly validating that `TradeKeeper` is initialized before invoking any of its methods.

Status: Resolved